EE 467 Lab 2: Breaking CAPTCHAs with Tensorflow / Keras

In lab 2, we are going to work on automatically breaking CAPTCHAs with deep learning! Originally, CAPTCHA stands for "Completely Automated Public Turing test to tell Computers and Humans Apart", and traditional CAPCTHAs serve as a great tool to stop spam bots and malicious crawlers. Today, as we have made huge progress in computer vision and deep learning, these CAPCTHAs are no longer unbreakable by computers. Now let's prove the correctness of the claim by ourselves!

As usual, please check if the helper library, `lab_2_helpers.py` and the extracted dataset directory, `captcha-images` exist under the same directory. Then, please install all libraries used for this lab:

In [1]:
```
%pip install matplotlib scikit-learn "opencv-python>4" imutils

# remove broken install
%pip uninstall -y torch torchvision torchaudio

# reinstall CPU-only PyTorch
%pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: opencv-python>4 in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: imutils in /usr/local/lib/python3.12/dist-packages (0.5.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Found existing installation: torch 2.10.0+cpu
Uninstalling torch-2.10.0+cpu:
  Successfully uninstalled torch-2.10.0+cpu
Found existing installation: torchvision 0.25.0+cpu
Uninstalling torchvision-0.25.0+cpu:
  Successfully uninstalled torchvision-0.25.0+cpu
Found existing installation: torchaudio 2.10.0+cpu
Uninstalling torchaudio-2.10.0+cpu:
  Successfully uninstalled torchaudio-2.10.0+cpu
Looking in indexes: https://download.pytorch.org/whl/cpu
Collecting torch
  Using cached https://download.pytorch.org/whl/cpu/torch-2.10.0%2Bcpu-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (29 kB)
Collecting torchvision
  Using cached https://download.pytorch.org/whl/cpu/torchvision-0.25.0%2Bcpu-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (5.4 kB)
Collecting torchaudio
  Using cached https://download.pytorch.org/whl/cpu/torchaudio-2.10.0%2Bcpu-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (6.9 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Using cached https://download.pytorch.org/whl/cpu/torch-2.10.0%2Bcpu-cp312-cp312-manylinux_2_28_x86_64.whl (188.9 MB)
Using cached https://download.pytorch.org/whl/cpu/torchvision-0.25.0%2Bcpu-cp312-cp312-manylinux_2_28_x86_64.whl (1.9 MB)
Using cached https://download.pytorch.org/whl/cpu/torchaudio-2.10.0%2Bcpu-cp312-cp312-manylinux_2_28_x86_64.whl (413 kB)
Installing collected packages: torch, torchvision, torchaudio
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
fastai 2.8.6 requires torch<2.10,>=1.10, but you have torch 2.10.0+cpu which is incompatible.
Successfully installed torch-2.10.0+cpu torchaudio-2.10.0+cpu torchvision-0.25.0+cpu
```

Next, we import all tools needed before starting:

In [2]:
```
import os, pickle, glob, math
from pprint import pprint

import cv2
import numpy as np
import imutils
from imutils import paths

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

from matplotlib import pyplot as plt
from matplotlib.gridspec import GridSpec

from sklearn.preprocessing import LabelBinarizer
from sklearn.model_selection import train_test_split

from lab_2_helpers import *
```

Preprocessing

Ground Truth Characters Extraction

As usual, we will start pre-processing stage by loading CAPTCHA images into the memory:

In [3]:
```
!tar -xJf captcha-images.tar.xz
```

In [4]:
```
# Dataset images folder
CAPTCHA_IMAGE_FOLDER = "./captcha-images"

# List of all the captcha images we need to process
captcha_image_paths = list(paths.list_images(CAPTCHA_IMAGE_FOLDER))
# Review image paths
pprint(captcha_image_paths[:10])
```

```
['./captcha-images/3F2L.png',
 './captcha-images/J3Q3.png',
 './captcha-images/2HKY.png',
 './captcha-images/3G9T.png',
 './captcha-images/B6RD.png',
 './captcha-images/6RTM.png',
 './captcha-images/2E8X.png',
 './captcha-images/2SBQ.png',
 './captcha-images/2NNC.png',
 './captcha-images/4NGH.png']
```

Note that for each image, its file name (without extension) happens to be its corresponding CAPTCHA text. Thus, we extract file names for all CAPTCHA images and save them as labels for future use:

In [5]:
```
def extract_captcha_text(image_path):
    """ Extract correct CAPTCHA texts from file name of images. """
    # Extract file name of image from its path
    # e.g. "./captcha-images/2A2X.png" -> "2A2X.png"
    image_file_name = os.path.basename(image_path)
    # Extract base name of image, omitting file extension
    # e.g. "2A2X.png" -> "2A2X"
    return os.path.splitext(image_file_name)[0]

captcha_texts = [extract_captcha_text(image_path) for image_path in captcha_image_paths]
# Review extraction results
pprint(captcha_texts[:10])
```

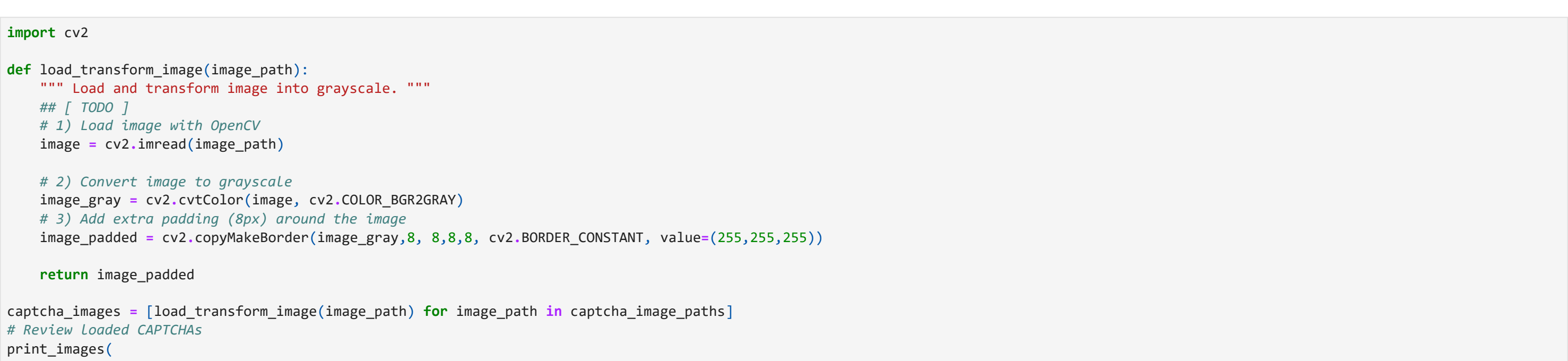
```
['3F2L', 'J3Q3', '2HKY', '3G9T', 'B6RD', '6RTM', '2E8X', '2SBQ', '2NNC', '4NGH']
```

Loading and Transforming Images

For the feature extraction stage, we are going to extract individual characters from these CAPTCHAs. This is done by looking for contours (bounding boxes) around characters, then cropping the CAPTCHAs such as only the contour areas are preserved. We begin feature extraction by loading and transforming images:

In [6]:
```
import cv2

def load_transform_image(image_path):
    """ Load and transform image into grayscale. """
    ## [ TODO ]
    # 1) Load image with OpenCV
    image = cv2.imread(image_path)

    # 2) Convert image to grayscale
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # 3) Add extra padding (8px) around the image
    image_padded = cv2.copyMakeBorder(image_gray,8, 8,8,8, cv2.BORDER_CONSTANT, value=(255,255,255))

    return image_padded

captcha_images = [load_transform_image(image_path) for image_path in captcha_image_paths]
# Review loaded CAPTCHAs
print_images(
    captcha_images[:10], n_rows=2, texts=captcha_texts[:10]
)
```

```
Figure(2000x500)
```



Next, we will split our dataset into train-validation set and test set. The former set will be used for training and validation in deep character classification model, while the latter will be used for testing our CAPTCHA recognition pipline end-to-end:

In [7]:
```
# Train-validation-test split seed
TVT_SPLIT_SEED = 31528476

# Perform split on CAPTCHA images as well as labels
captcha_images_tv, captcha_images_test, captcha_texts_tv, captcha_texts_test = train_test_split(
    captcha_images, captcha_texts, test_size=0.2, random_state=TVT_SPLIT_SEED
)

print("Train-validation:", len(captcha_texts_tv))
print("Test:", len(captcha_texts_test))
```

```
Train-validation: 908
Test: 228
```

Bounding Box Extraction

It's now time to perform the most important feature extraction step: finding contours and extracting characters. Contours can be explained simply as a curve joining all the continuous points (along the boundary), having same color or intensity. It is useful for shape analysis and object detection and recognition. For our task however, we are **more interested in the bounding boxes around characters**, since these are the part of images we will be used for character classification:

Here are the steps we are going to perform:

- Find contours around characters in CAPTCHAs.
- Get the position and size for each corresponding bounding box.
- If a bounding box is too wide (width-to-height ratio larger than 1.25), chances are that we have bounded two letters in a single bounding box. In this case, split the bounding box vertically from the center into two.
- Store all bounding boxes for the CAPTCHA.
- If there aren't 4 bounding boxes for the CAPTCHA, ignore it since our character extraction process must have run into problems in this case.
- Sort the bounding boxes by their X coordinates, so that they match the order corresponding letters occur.
- For each bounding box, extract corresponding region of the image, and store it as an instance of corresponding character at `${CHAR_IMAGE_FOLDER}/{letter}/{count}.png`.

After these steps, we have transformed CAPTCHA images into images of single character. This simplifies our task since now our model only needs to deal with classification (from character image to character itself) rather than also dealing with detection (finding and extracting charatcers).

In [1]:
```
# Character images folder template
CHAR_IMAGE_FOLDER = f"./char-images-{TVT_SPLIT_SEED}"

def extract_chars(image):
    """ Find contours and extract characters inside each CAPTCHA. """
    # Threshold image and convert it to black-white
    image_bw = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)[1]
    # Find contours (continuous blobs of pixels) the image
    contours = cv2.findContours(image_bw, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)[0]

    char_regions = []
    # Loop through each contour
    for contour in contours:
        # Get the rectangle that contains the contour
        x, y, w, h = cv2.boundingRect(contour)

        # Compare the width and height of the bounding box,
        # detect if there are letters conjoined into one chunk
        if w / h > 1.25:
            # Bounding box is too wide for a single character
            # Split it in half into two letter regions
            half_width = int(w / 2)
            char_regions.append((x, y, half_width, h))
            char_regions.append((x + half_width, y, half_width, h))
        else:
            # Only a single letter in contour
            char_regions.append((x, y, w, h))

    # Ignore image if less or more than 4 regions detected
    if len(char_regions)!=4:
        return None
    # Sort regions by their X coordinates
    char_regions.sort(key=lambda x: x[0])

    # Character images
    char_images = []
    # Save each character as a single image
    for x, y, w, h in char_regions:
        # Extract character from image with 2px margin
        char_image = image[y - 2:y + h + 2, x - 2:x + w + 2]
        # Save character images
        char_images.append(char_image)

    # Return character images
    return char_images

def save_chars(char_images, captcha_text, save_dir, char_counts):
    """ Save character images to directory. """
    for char_image, char in zip(char_images, captcha_text):
        # Get the folder to save the image in
        save_path = os.path.join(save_dir, char)
        os.makedirs(save_path, exist_ok=True)

        # Write letter image to file
        char_count = char_counts.get(char, 1)
        char_image_path = os.path.join(save_path, f"{char_count}.png")
        cv2.imwrite(char_image_path, char_image)

        # Update count
        char_counts[char] = char_count+1

# Force character extraction even if results are already available
FORCE_EXTRACT_CHAR = False

char_counts = {}
# Extract and save images for characters
if FORCE_EXTRACT_CHAR or not os.path.exists(CHAR_IMAGE_FOLDER):
    for captcha_image, captcha_text in zip(captcha_images_tv, captcha_texts_tv):
        # Extract character images
        char_images = extract_chars(captcha_image)
        # Skip if extraction failed
        if char_images is None:
            continue
        # Save character images
        save_chars(char_images, captcha_text, CHAR_IMAGE_FOLDER, char_counts)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 2
      1 # Character images folder template
----> 2 CHAR_IMAGE_FOLDER = f"./char-images-{TVT_SPLIT_SEED}"
      4 def extract_chars(image):
      5     """ Find contours and extract characters inside each CAPTCHA. """

NameError: name 'TVT_SPLIT_SEED' is not defined
```

Label Encoding

During the training stage, we are going to load character images from previous stages as features and generate corresponding labels from their path. We will then rescale features, one-hot encode labels (occurred characters) and save labels to an external file.

In [9]:
```
# Path of occurred characters (labels)
LABELS_PATH = "./labels.pkl"

def make_feature(image):
    """ Process character image and turn it into feature. """
    # Resize letter to 20*20
    image_resized = resize_to_fit(image, 20, 20)
    # Add extra dimension as the only channel
    feature = image_resized[..., None]

    return feature

def make_feature_label(image_path):
    """ Load character image and make feature-label pair from image path. """
    # Load image and make feature
    feature = make_feature(cv2.imread(image_path, cv2.COLOR_BGR2GRAY))
    # Extract label based on the directory the image is in
    label = image_path.split(os.path.sep)[-2]

    return feature, label

# Make features and labels from character image paths
features_tv, labels_tv = unzip((
    make_feature_label(image_path) for image_path in paths.list_images(CHAR_IMAGE_FOLDER)
))

# Scale raw pixel values into range [0, 1]
features_tv = np.array(features_tv, dtype="float")/255
# Convert labels into one-hot encodings
lb = LabelBinarizer()
labels_one_hot_tv = lb.fit_transform(labels_tv)
# Number of classes
n_classes = len(lb.classes_)

# Further split the training data into training and validation set
X_train, X_vali, y_train, y_vali = train_test_split(
    features_tv, labels_one_hot_tv, test_size=0.25, random_state=955996
)
# Save mapping from labels to one-hot encoding
with open(LABELS_PATH, "wb") as f:
    pickle.dump(lb, f)
```

Training

Next, we build a Convolutional Neural Network (CNN) as our classification model with Tensorflow / Keras. The structure of the neural network is shown below:

After building the neural network, we train it and save weights.

In [11]:
```
# Batch size
BATCH_SIZE = 32
# Number of epochs
N_EPOCHS = 10

# Path of model weights file
MODEL_WEIGHTS_PATH = "./captcha-model.pt"
# Force training even if weights are already available
FORCE_TRAINING = True

import torch
import torch.nn as nn
import torch.optim as optim

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# ------------------------
# Build neural network
# ------------------------

model = nn.Sequential(
    # First convolution block
    # (*, 1, 20, 20) -> (*, 8, 20, 20) -> (*, 8, 10, 10)
    nn.Conv2d(1, 8, kernel_size=5, stride=1, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),

    # Second convolution block
    # (*, 8, 10, 10) -> (*, 50, 10, 10) -> (*, 50, 5, 5)
    nn.Conv2d(8, 50, kernel_size=5, stride=1, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),

    # Flatten
    nn.Flatten(),

    # Fully connected layers
    nn.Linear(5 * 5 * 50, 500),
    nn.ReLU(),

    # Output layer (NO softmax in PyTorch)
    nn.Linear(500, 32)
).to(device)

# ------------------------
# Loss + optimizer
# ------------------------

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())

# ------------------------
# Data conversion
# ------------------------
# TF: (N, 20, 20, 1)
# PT: (N, 1, 20, 20)

X_train = torch.tensor(X_train).permute(0, 3, 1, 2).float()
X_vali  = torch.tensor(X_vali).permute(0, 3, 1, 2).float()

# labels must be class indices (not one-hot)
if len(y_train.shape) > 1:
    y_train = np.argmax(y_train, axis=1)
    y_vali  = np.argmax(y_vali, axis=1)

y_train = torch.tensor(y_train).long()
y_vali  = torch.tensor(y_vali).long()

# ------------------------
# Train / load
# ------------------------

if FORCE_TRAINING or not os.path.exists(MODEL_WEIGHTS_PATH):

    for epoch in range(N_EPOCHS):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for i in range(0, len(X_train), BATCH_SIZE):
            x_batch = X_train[i:i+BATCH_SIZE].to(device)
            y_batch = y_train[i:i+BATCH_SIZE].to(device)

            optimizer.zero_grad()
            outputs = model(x_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            preds = outputs.argmax(dim=1)
            correct += (preds == y_batch).sum().item()
            total += y_batch.size(0)

        acc = correct / total
        print(f"Epoch [{epoch+1}/{N_EPOCHS}] Loss: {running_loss:.4f} Acc: {acc:.4f}")

    torch.save(model.state_dict(), MODEL_WEIGHTS_PATH)

else:
    model.load_state_dict(torch.load(MODEL_WEIGHTS_PATH, map_location=device))

# ------------------------
# Model summary
# ------------------------

print(model)
```

```
Epoch [1/10] Loss: 208.1783 Acc: 0.3281
Epoch [2/10] Loss: 32.9474 Acc: 0.9095
Epoch [3/10] Loss: 12.2028 Acc: 0.9719
Epoch [4/10] Loss: 7.0196 Acc: 0.9858
Epoch [5/10] Loss: 4.2747 Acc: 0.9914
Epoch [6/10] Loss: 3.0058 Acc: 0.9936
Epoch [7/10] Loss: 2.2224 Acc: 0.9948
Epoch [8/10] Loss: 1.3013 Acc: 0.9970
Epoch [9/10] Loss: 0.4359 Acc: 0.9996
Epoch [10/10] Loss: 0.2062 Acc: 1.0000
Sequential(
  (0): Conv2d(1, 8, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (1): ReLU()
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (3): Conv2d(8, 50, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (4): ReLU()
  (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=1250, out_features=500, bias=True)
  (8): ReLU()
  (9): Linear(in_features=500, out_features=32, bias=True)
)
```

Evaluation

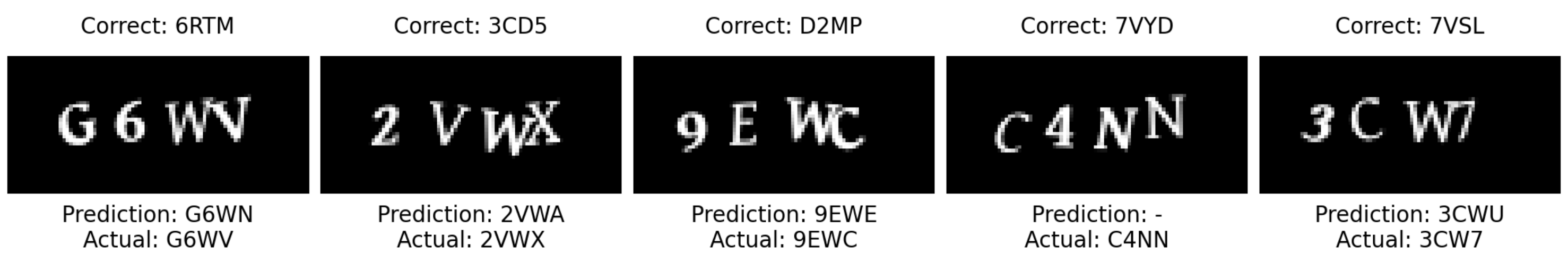
During the training part, we have validated the performance of our neural network model on images of single characters. Now it's time to test and evaluate CAPTCHAs from the beginning to the end. First, we will need to build the pipeline for CAPTCHA character prediction:

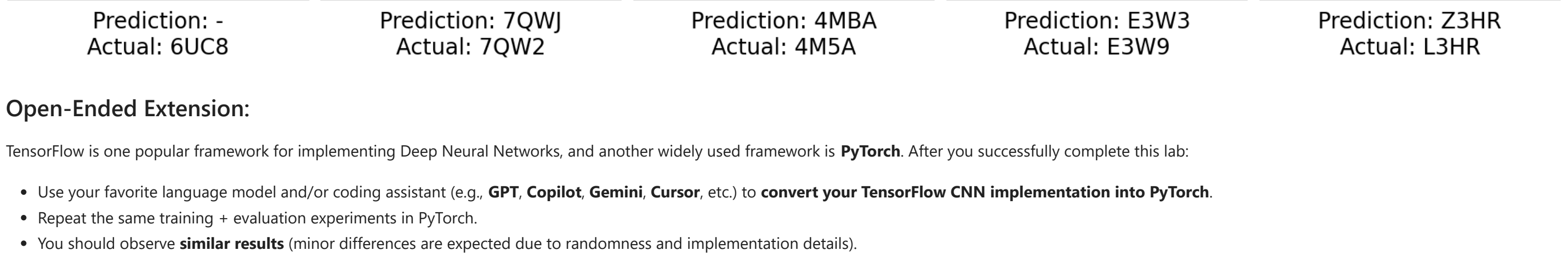
In [18]:
```
# Load labels from file (so we can translate model predictions to actual letters)
with open(LABELS_PATH, "rb") as f:
    lb = pickle.load(f)

# Test our pipeline (and model) with the test set.
# However, you'd want to replace this with some random CAPTCHAs in the real world.

# Dummy character images
DUMMY_CHAR_IMAGES = np.zeros((4, 20, 20, 1))

# Indices of CAPTCHAs on which extractions failed
extract_failed_indices = []
# Extracted character images
char_images_test = []

# Extract character images and make features
for i, captcha_image in enumerate(captcha_images_test):
    # Extract character images
    char_images = extract_chars(captcha_image)

    if char_images:
        char_images_test.extend(char_images)
    # Use dummy character images as placeholder if extraction failed
    else:
        extract_failed_indices.append(i)
        char_images_test.extend(DUMMY_CHAR_IMAGES)

# Make features for character images
features_test = [make_feature(char_image) for char_image in char_images_test]

# Scale raw pixel values into range [0, 1]
features_test = np.array(features_test, dtype="float32") / 255.0

# Convert to PyTorch tensor
# TensorFlow: (N, 20, 20, 1)
# PyTorch:    (N, 1, 20, 20)
features_test = torch.tensor(features_test).permute(0, 3, 1, 2)

# Predict labels with neural network (PyTorch equivalent of model.predict)
model.eval()
with torch.no_grad():
    outputs = model(features_test)
    preds_test = outputs.argmax(dim=1).cpu().numpy()

# Convert predicted class indices back to characters
preds_test = lb.classes_[preds_test]

# Group all 4 characters for the same CAPTCHA
preds_test = ["".join(chars) for chars in group_every(preds_test, 4)]

# Update result for CAPTCHAs on which extractions failed
for i in extract_failed_indices:
    preds_test[i] = "-"
```

Now, we can compute the accuracy of our pipeline, as well as taking a look at correct and incorrect CAPTCHA text predictions:

In [19]:
```
# Number of CAPTCHAs to display
N_DISPLAY_SAMPLES = 10

# Number of test CAPTCHAs
n_test = len(captcha_texts_test)
# Number of correct predictions
n_correct = 0

# Indices of correct predictions
correct_indices = []
# Indices of incorrect predictions
incorrect_indices = []

for i, (pred_text, actual_text) in enumerate(zip(preds_test, captcha_texts_test)):
    if pred_text==actual_text:
        ## [ TODO ]
        # 1) Update number of correct predictions
        n_correct += 1
        # 2) Collect index of correct prediction
        if len(correct_indices)<N_DISPLAY_SAMPLES:
            correct_indices.append(i)
    else:
        # 3) Collect index of incorrect prediction
        if len(incorrect_indices)<N_DISPLAY_SAMPLES:
            incorrect_indices.append(i)

# Show number of total / correct predictions and accuracy
print("# of test CAPTCHAs:", n_test)
print("# correctly recognized:", n_correct)
print("Accuracy:", n_correct/n_test, "\n")

# Show all correct predictions
print_images(
    [captcha_images_test[i] for i in correct_indices],
    texts=[f"Correct: {captcha_texts_test[i]}" for i in correct_indices],
    n_rows=2
)

# Show all incorrect predictions
print_images(
    [captcha_images_test[i] for i in incorrect_indices],
    texts=[
        f"Prediction: {preds_test[i]}\nActual: {captcha_texts_test[i]}" \
        for i in incorrect_indices
    ],
    n_rows=2,
    fig_size=(20, 6),
    text_center=(0.5, -0.25)
)
```

```
# of test CAPTCHAs: 228
# correctly recognized: 218
Accuracy: 0.956140350877193

Figure(2000x500)
Figure(2000x600)
```





Open-Ended Extension:

TensorFlow is one popular framework for implementing Deep Neural Networks, and another widely used framework is **PyTorch**. After you successfully complete this lab:

- Use your favorite language model and/or coding assistant (e.g., **GPT**, **Copilot**, **Gemini**, **Cursor**, etc.) to **convert your TensorFlow CNN implementation into PyTorch**.
- Repeat the same training + evaluation experiments in PyTorch.
- You should observe **similar results** (minor differences are expected due to randomness and implementation details).

✅ **Submission Requirement:**
Include this as a **separate notebook** in your lab-submission GitHub repository named:

`Breaking-CAPTCHAS-Pytorch.ipynb`

In [ ]:

References

1. How to break a CAPTCHA system in 15 minutes with Machine Learning: https://medium.com/@ageitgey/how-to-break-a-captcha-system-in-15-minutes-with-machine-learning-dbebb035a710
2. CaptchaSolver Jupyter Notebook: https://github.com/BenjaminWegener/CaptchaSolver
3. Keras Tutorial: The Ultimate Beginner's Guide to Deep Learning in Python: https://elitedatascience.com/keras-tutorial-deep-learning-in-python
4. Keras API reference: https://keras.io/
5. Tensorflow API reference: https://www.tensorflow.org/api_docs